11 Files and Collections

Note 1: In this set of exercises, we are using the **Path** class from the *pathlib* module. Please see the course material for further explanation and details.

Note 2: Viope does not allow you to include more than one Python file *of your own* in your solution when you are testing your program in Viope. Therefore, we cannot generalise our own file handling logic and place it in a separate module. Otherwise, we would do this in some of these exercises.

Note 3: We are not using Python's *csv* module. Instead, we are focusing on data structures.

Files Read

Create a program called **files_read**. The program should first input the name of a text file from the user. Then the program should read the file content and print it as it is. No formatting needed here.

Please read the content of the text file using the **Path** class from the **pathlib** module

If the file is not found, the program should print "**The file** *xxxx.yyy* **is not found**" (here *xxxx.yyy* is the inputted file name).

You can download the file ***files_keywords.txt*** from the Moodle page and use it in this exercise when you are running your program. Of course, you can test your program using any text file on your PC.

Example output:

```
Enter file name: files_keywords.txt

Python 3.10 keywords
--------------------
False       await       else        import      pass
None        break       except      in          raise
True        class       finally     is          return
and         continue    for         lambda      try
as          def         from        nonlocal    while
assert      del         global      not         with
async       elif        if          or          yield
```

Example output:

```
Enter file name: somefile.txt

The file somefile.txt is not found
```

Note: In Viope, the text files used in test cases are in the same folder where Viope saves your own Python source text. This folder is also the current work folder for running Python programs.

Files City Bikes 1

Create a program called **files_city_bikes_1**. The program should read city bike usage data from a CSV file and print basic statistics on the bike rides as shown in the example output.

The file ***city_bike_rides_2021-06-30.csv*** contains the city bike usage data to be used in this exercise. Please download the file from the Moodle page. **NB!** When you open the file in a *text editor*, you can *see the **column headers** on the first line of the file*.

Hint: For your local testing, you can make a copy of the file ***city_bike_rides_2021-06-30.csv*** and give it a shorter name 😀

INSTRUCTION (*To be carefully followed in this exercise*)

The program should have the following three functions: **main**, **load_file**, **show_statistics**

The **load_file** function should

- Take a CSV file name as argument.
- Read the content of the CSV file using the **Path** class from the **pathlib** module
- Create a list that contains all other lines but not the first line read from the CSV file
- Return a reference to the list or return **None** if the CSV file is not found.

The **show_statistics** function should

- Take a reference to a list (that contains the bike ride data) as argument.
- Compute and display bike ride statistics as shown in the example output.

The **main** function should

- Input the name of a CSV file from the user. If the file is not found, the program should print "**The file** *xxxx.yyy* **is not found**" (here *xxxx.yyy* is the inputted file name).
- Call the **load_file** function.
- Print "**The file** *xxxx.yyy* **is not found**" (here *xxxx.yyy* is the inputted file name) if the file is not found.
- If the file is found, call the **show_statistics** function.

Example output:

```
Enter file name: city_bike_rides_2021-06-30.csv

43578 kilometers on 18272 bike rides
Average distance: 2.4 kilometers
Average duration: 15 minutes
```

Example output:

```
Enter file name: somefile.csv

The file somefile.csv is not found
```

Files Postcodes 1

Create a program called **files_postcodes_1**. The program should input a CSV file name from the user, read post-office data from a CSV file, input a ***postcode*** from the user and print the post-office name in Finnish and Swedish as shown in the example output.

The file ***postcodes.csv*** contains the post-office data to be used in this exercise. Please download the file from the Moodle page. **NB!** When you open the file in a *text editor*, you can see the row structure.

INSTRUCTION (*To be carefully followed in this exercise*)

The program should have a function called **load_data** that should

- Take a CSV file name as argument.
- Read the content of the CSV file using the **Path** class from the **pathlib** module
- Create a ***dictionary*** and save facts about each post-office to the dictionary
- Return a *reference to the dictionary* or return **None** if the CSV file is not found.

The **main** function should

- Input the name of the CSV file
- Call the **load_data** function
- Print the number of rows in the file, or "**The file** *xxxx.yyy* **is not found**" if the file is not found (here *xxxx.yyy* is the inputted file name)
- If the file is found, input a postcode once and print facts about the post-office as shown in the example output, or "**Unknown postcode**" if the postcode does not exist.

Example output:

```
Enter postcode file name: postcodes.csv
4997 rows

Enter postcode: 00520
00520 HELSINKI
00520 HELSINGFORS
```

Example output:

```
Enter postcode file name: somefile.csv

The file somefile.csv is not found
```

Example output:

```
Enter postcode file name: postcodes.csv
4997 rows

Enter postcode: 12345
Unknown postcode
```

Files Postcodes 2

Create a program called **files_postcodes_2**. The program should input a CSV file name from the user and read post-office data from a CSV file. Then the program should input a ***place name*** from the user and print the postcodes and post-office names ***in the same language as the user entered the place name***.

INSTRUCTION (*to be carefully followed in this exercise*)

- You should use the same file (***postcodes.csv***) in this exercise as you used in the previous exercise.
- You should copy the **load_file** function from the previous exercise and use it in this exercise.

The **main** function should

- Print "**The file postcodes.csv is not found**" if the file is not found
- If the file is found, input a place name once and print postcodes and post-office names as shown in the example output, or "**No post-office found**" if the place name is not found.

Example output:

```
Enter place name: Kalajoki

85100 KALAJOKI
85101 KALAJOKI
```

Example output:

```
Enter place name: Närpes

64200 NÄRPES
64201 NÄRPES
```

Example output:

```
Enter place name: London

No postoffice found
```

Files City Bikes 2 (*This is a grade level 5 exercise*)

Create a program called **files_city_bikes_2**. The program is an extended version of the **files_city_bikes_1** program.

The files ***city_bike_rides_small.csv*** and ***city_bike_rides_2021-06-30.csv*** contain the city bike usage data to be used in this exercise. Please download the files from the Moodle page.

INSTRUCTION (*To be carefully followed in this exercise*)

In addition to the previous output, the program should print the ***names of those bike stations who have the highest number of departures.*** The bike station names should be printed in ascending order.

You should extend your previous code by writing a function named **show_max_departures**. The function should

- Take a reference to a list (that contains the bike ride data) as argument.
- Compute and print names of the bike station(s) who has/have the highest number of departures. The bike station names should be printed in ascending order.

The main function should call the **show_max_departures** function.

Example output:

```
Enter file name: city_bike_rides_2021-06-30.csv

43578 kilometers on 18272 bike rides
Average distance: 2.4 kilometers
Average duration: 15 minutes

Maximum number of departures from a bike station:
Itämerentori (314 departures)
```

Example output:

```
Enter file name: city_bike_rides_small.csv

71 kilometers on 22 bike rides
Average distance: 3.2 kilometers
Average duration: 17 minutes

Maximum number of departures from a bike station:
Agronominkatu (4 departures)
Pasilan asema (4 departures)
Ympyrätalo (4 departures)
```

Example output:

```
Enter file name: somefile.txt

The file somefile.txt is not found
```